Neural Ledger

Technical Documentation

Architecture, algorithms and security design

An AI cash-flow copilot with deterministic authority
FinTechathon 2026 · International Track (Agent Wallet) · Topic A: Personal Finance Assistant

Role	Owner
A · Finance core	Bruno Caro
B · Agent / AI	Luis-Felipe
C · Platform	Baptiste
D · Frontend	Vicente Ramírez
E · Quality and submission	Joaquín Parraud

Date	19 October 2026
Document version	1.0
Software version	0.1.0 (`backend/pyproject.toml`)
Code described	commit `58ff160` on branch `security/option-b-hardening`

This document describes the system as implemented at the commit named above. Every claim points to the file or function that implements it; where the Markdown documentation in `docs/` disagrees with the code, the code is followed and the difference is noted in a *Docs vs code* paragraph and collected in Section 8. The repository is public, and all data in it is synthetic.

Contents

1 Overview **3**
1.1 Design principles . . . 3
1.2 Scope and sources of this document . . . 3

2 System architecture **4**
2.1 Layers and the dependency rule . . . 4
2.2 Runtime modes and containers . . . 4
2.3 HTTP surface and identity . . . 4
2.4 A chat turn . . . 7
2.5 The confirmation contract . . . 8

3 Data model **9**
3.1 The financial model . . . 9
3.2 Statements, transactions and actions . . . 9
3.3 Users, sessions and the audit trail . . . 10
3.4 Envelopes and snapshot ids . . . 10
3.5 Persistence schema . . . 10

4 Financial core algorithms **11**
4.1 Estimating the model (stage 1) . . . 11
4.2 Forecast at pay-cycle troughs . . . 12
4.3 Protected surplus . . . 12
4.4 Financial capacity . . . 12
4.5 Sizing a savings plan . . . 13
4.6 Simulating a purchase . . . 13
4.7 Importing a statement . . . 14
4.8 The response verifier . . . 14
4.9 The audit chain and erasure . . . 16

5 Key design decisions **16**
ADR-001 · Clean architecture with a standard-library core . . . 17
ADR-003 · The application orchestrates; the language model is an adapter . . . 17
ADR-004 · A decision-model port of closed questions, in a configurable chain . . . 17
ADR-006 · Offline mode is a requirement . . . 17
ADR-008 · HMAC-signed authorizations and a hash-chained audit log . . . 17
ADR-013 · Content-addressed snapshot ids and a parameter lock . . . 17
ADR-020 · Reconciliation is the authority over the reader . . . 18
ADR-028 · Personal data is removed from a statement once, at import . . . 18
ADR-037 · The user's own liquidity floor . . . 18
ADR-038 · Option B: open sign-up, explicit sandbox, everything sealed, deletable accounts 18
Open decisions . . . 18

6 Security design **19**
6.1 Threat model . . . 19
6.2 Trust boundaries . . . 19
6.3 Agent permissions . . . 19
6.4 Accounts, sessions and the sandbox notice . . . 20
6.5 Data at rest . . . 20
6.5.1 Key hierarchy . . . 20
6.5.2 Migration and key rotation . . . 21
6.6 Account deletion . . . 21

6.7 Evidence 24

7 Privacy and Law 21.719 **24**
7.1 What is processed, where and for how long 24
7.2 Duties and rights 24
7.3 The sandbox posture 25

8 Where the Markdown documentation and the code disagree **25**

1 Overview

Neural Ledger is a personal cash-flow copilot. A user asks, in Spanish or English, whether a purchase fits, how much money is really available, and what to do with a surplus; the copilot answers with figures computed from the user's financial model and, inside limits the user confirms, creates a recurring rule in a *sandbox* bank. No real money moves. The product's central distinction is that a balance is not the same as the money that is available, and that neither is the same as a *protected surplus*: the amount that could leave the account today without any future pay-cycle trough falling below a liquidity floor (Section 4.3). The README's example: Camila's bank shows 920,000 CLP, but her protected surplus is 20,000 CLP because three installment plans (*cuotas*) overlap in December.

1.1 Design principles

Five principles, stated in `CLAUDE.md` and enforced by tests, shape everything that follows.

Deterministic authority.
Money is computed, and actions are authorized, only by domain code. A language model interprets the user and writes prose; it never sets an amount, a verdict or a permission. Execution terms are read from a stored, signed authorization, never from model text, and only the user can confirm an action (Section 2.5).

Every number is a fact.
Each financial tool returns an *envelope* whose `facts` list every number the agent may say and whose `warnings` list every assumption it must mention. A deterministic verifier rejects a reply that breaks either rule (Section 4.8).

Contracts first.
JSON Schemas in `contracts/` define tools, outputs, actions, audit events and HTTP bodies. The backend validates against them in tests and the web client generates its TypeScript types from them.

Offline is a requirement.
Every port has an adapter that needs no network and no key, so the whole demo, the test suite and `docker compose up` run offline (Section 2.2).

Exact money.
Money is an integer in minor units; ratios stay exact (`Fraction`, `Decimal`) until the final rounding (`domain/shared/numbers.py`). Money is never called "safe".

1.2 Scope and sources of this document

The document consolidates five Markdown files into one description: `docs/ARCHITECTURE.md` (where code goes), `docs/FINANCIAL_CORE.md` (the engine), `docs/DECISIONS.md` (the decision log), `docs/SECURITY.md` and `docs/PRIVACY.md`. It describes the code at commit `58ff160`, which includes the security hardening called *option B* (ADR-038): open sign-up with a sandbox notice, every personal field sealed at rest, the data key kept off the data volume, and account deletion. A companion document, the *Security self-assessment report*, evaluates the same system against its threats and lists the residual risks.

Table 1 is a map of the repository.

Path	Contents
backend/src/neural_ledger/domain	Pure rules: finance, permissions, actions, audit chain, agent verification, privacy redaction, statements, user records
backend/src/neural_ledger/application	Use cases, services and the ports (interfaces) the use cases need
backend/src/neural_ledger/adapters	Inbound (HTTP with FastAPI, CLI) and outbound (persistence, models, bank, readers) implementations
backend/src/neural_ledger/bootstrap	Configuration, key resolution, migration and the composition root (container.py)
backend/tests	Architecture, golden, property, security, suitability, contract, adapter and HTTP tests
contracts/	JSON Schemas (draft 2020-12): the source of truth for every interface
data/	Synthetic personas (Camila, Diego) and generated ledgers
web/	React and TypeScript client; renders API facts and never computes money
docker-compose*.yml, deploy/	Local and deployed container setup; the data key secret

Table 1: Repository map.

2 System architecture

2.1 Layers and the dependency rule

The backend is a clean (hexagonal) architecture in four packages (Figure 1). The rule is one-way: adapters depend on application, which depends on domain; bootstrap wires concrete adapters into use cases and nothing imports it. domain and application import only the Python standard library: no FastAPI, no SDK, no clock, no randomness, no file or network access. tests/architecture/test_dependency_rule.py parses every module's imports and fails the build on any violation; FastAPI may appear only in adapters/inbound/http.

Ports are typing.Protocol classes in application/ports.py. Each has at least one adapter that works offline, so swapping a provider means one adapter file and one registry entry in bootstrap/container.py (Table 2).

2.2 Runtime modes and containers

Four independent settings decide how an instance runs (bootstrap/config.py); GET /health reports the first two so that a demo never runs in the wrong one.

docker-compose.yml is the submission and CI path: it runs offline, in accounts mode, with SQLite on the ledger-data volume, and needs no key; without one the API creates a key file next to the database and prints a warning. A deployment adds docker-compose.data-key.yml, which mounts the data key as a Docker secret at /run/secrets/nl_data_key and sets NL_DEPLOYMENT=deployed. The public demo's own override (no host ports, Cloudflare Tunnel, security headers in nginx) lives in deploy/docker-compose.deploy.yml on the branch ops/deploy-neural-ledger, which is not merged at the commit described here.

2.3 HTTP surface and identity

adapters/inbound/http/app.py is thin: parse the request, call one use case, serialize with to_primitive. In accounts mode the pure ASGI middleware IdentityGuard (identity.py) runs before any route and before any body is read, in this order:

1. any X-User-Id header, on any route but GET /health, is answered 400 IDENTITY_HEADER_REJECTED;

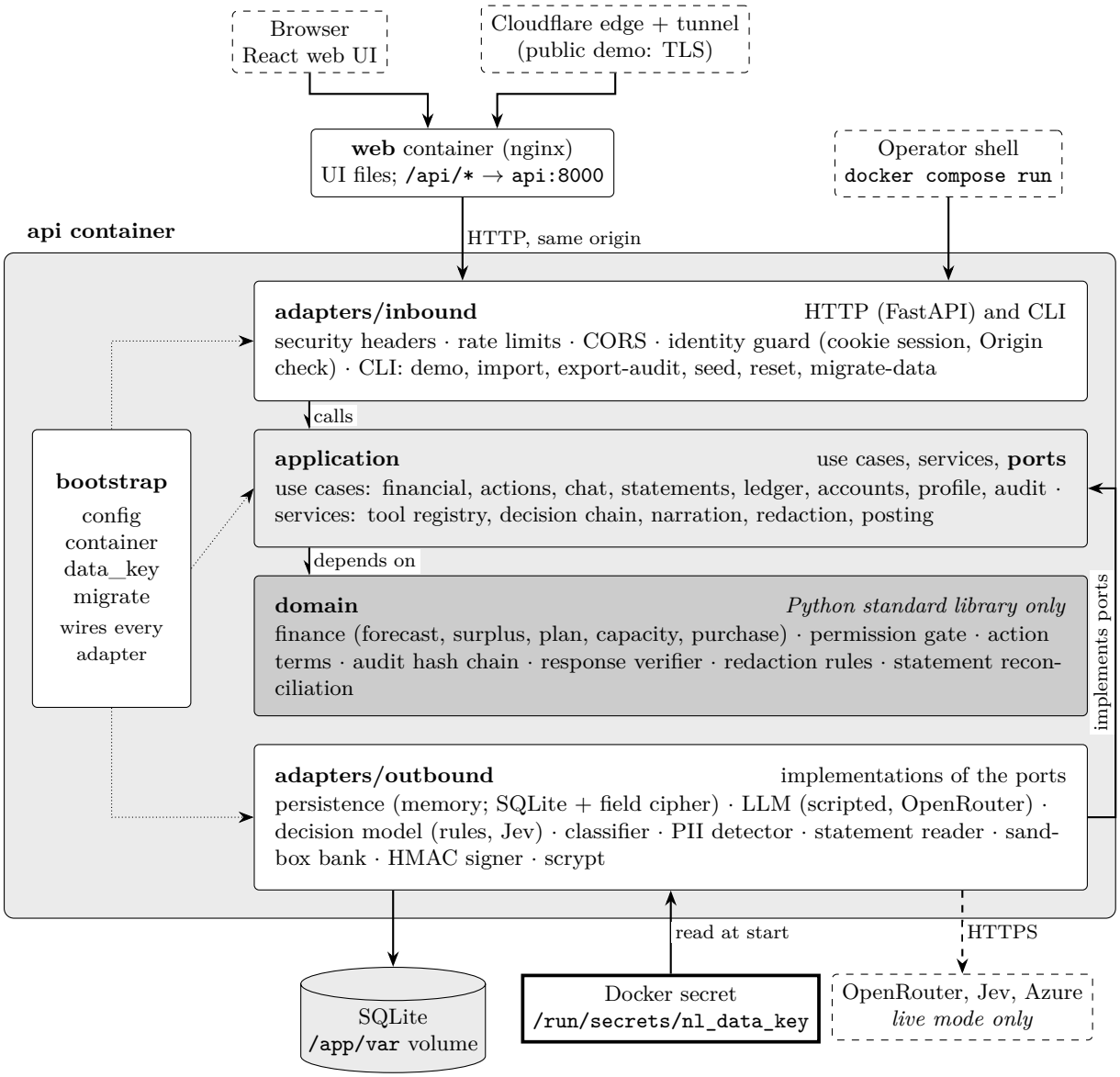


Figure 1: Components. The web container serves the client and proxies `/api`; the api container holds the four layers. Solid arrows inside the api are the dependency rule; dotted arrows are wiring by `bootstrap`. In a deployment the data key arrives as a Docker secret, separate from the data volume.

Port	Offline adapter	Other adapters
`LanguageModel`	`ScriptedLanguageModel`	OpenRouter; Anthropic (deprecated, ADR-027), behind a fallback to the scripted model
`DecisionModel`	`RulesDecisionModel` (keyword rules)	Jev through OpenRouter or TypeSafe, first in `NL_DECISION_CHAIN`
`TransactionClassifier`	naive Bayes over character n-grams	Jev, in a fallback chain (ADR-032)
`PersonalDataDetector`	`NullDetector` (rules only)	Azure AI Language (ADR-028)
`FinancialModelSource`	persona fixtures; ledger estimators	a user's own statements (ADR-029)
`StatementStore`, `TransactionStore`, `ActionRepository`, `AuditLog`, `UserAccountStore`, `SessionStore`, `UserProfileStore`	in memory	SQLite, every personal payload sealed (Section 6.5)
`AccountWrites`, `ProfileWrites`, `TransactionWrites`, `AccountErasure`	in memory, all locks held	SQLite, one transaction each
`SandboxBank`	`SimulatedSandboxBank`	none: no real bank is ever called
`Signer`, `PasswordHasher`, `TokenGenerator`	HMAC-SHA256, scrypt, `secrets`	none

Table 2: Ports and their adapters (`application/ports.py`, `bootstrap/container.py`).

Setting	Values	Effect
providers (`mode`)	offline, live	*Live* as soon as any setting sends data to a provider: `NL_LLM` other than `scripted`, `jev` in `NL_DECISION_CHAIN` or `NL_CLASSIFIER`, or `NL_PII_DETECTOR=azure`. Live mode refuses the development HMAC secret and, with SQLite, a missing data key.
`NL_AUTH`	demo, accounts	*demo*: the `X-User-Id` header names a persona, profiles are read-only, no auth routes (local runs and tests only). *accounts*: sign-up, sign-in and an HttpOnly session cookie; the header is rejected.
`NL_PERSISTENCE`	memory, sqlite	In-process stores, or one SQLite file at `NL_SQLITE_PATH`.
`NL_DEPLOYMENT`	local, deployed	*deployed* refuses to start without a data key from a secret or the environment, with a key on the data volume, while the old `<db>.key` exists, or with the development HMAC secret (Section 6.5.1).

Table 3: Runtime settings.

2. an unsafe method (POST, PUT, PATCH, DELETE) needs an `Origin` listed in `NL_APP_ORIGINS` or, without an Origin, a JSON body, which a cross-site HTML form cannot send (`403 ORIGIN_REJECTED`);

3. every route outside `PUBLIC_ROUTES` (health, chain verification, sign-up, sign-in, demo session) needs a valid `nl_session` cookie (`401 UNAUTHENTICATED`); the resolved `Session` is the only source of the user id.

Sessions last twelve hours from creation and never slide; the cookie is `HttpOnly`, `SameSite=Strict`, `Path=/`, and `Secure` over HTTPS. Only the SHA-256 digest of the token is stored. Outside the guard, `RateLimitMiddleware` counts per caller and per client IP in one-minute windows; `SecurityHeadersMiddleware` adds `X-Content-Type-Options: nosniff` and `Cache-Control: no-store` to every response.

Budget	Matches	Per caller	Per IP
statements	`POST /v1/statements`	10	30
chat	`POST /v1/chat`	20	60
auth	`POST /v1/auth/*`	10	10
account	`DELETE /v1/account`	5	10
api	any other `/v1/*`	300	900

Table 4: Rate limits per minute (`DEFAULT_LIMITS` in `rate_limit.py`); the first matching budget applies, and over it the answer is 429 with `Retry-After`.

2.4 A chat turn

The application, not the language model, owns the flow of a turn (ADR-003, `application/use_cases/chat.py`, Figure 2). The message is cut at 4,000 characters. A decision model (the rules, or Jev first and the rules behind it) classifies the raw text, independently of the language model, into an intent with a confidence and an injection flag. A deterministic gate (`domain/agent/policy.py`, policy `ap-0.2`) then decides: suspected injection is refused; a confident (at least 0.80) out-of-scope intent is refused; a confidence between 0.55 and 0.80 gets a clarifying question and no tool; below 0.55, or when no decision model answered, the turn is read-only (tiers T0 and T1, without the stateful proposal tools); at 0.80 or above, tiers up to T2 are allowed. The language model chooses among the tools of the allowed tiers, up to eight rounds. Its reply is checked by the verifier; an honest mistake (an unsupported number or a missing warning) may be repaired once, and anything else is replaced by a deterministic narration built from the facts. Every step is audited.

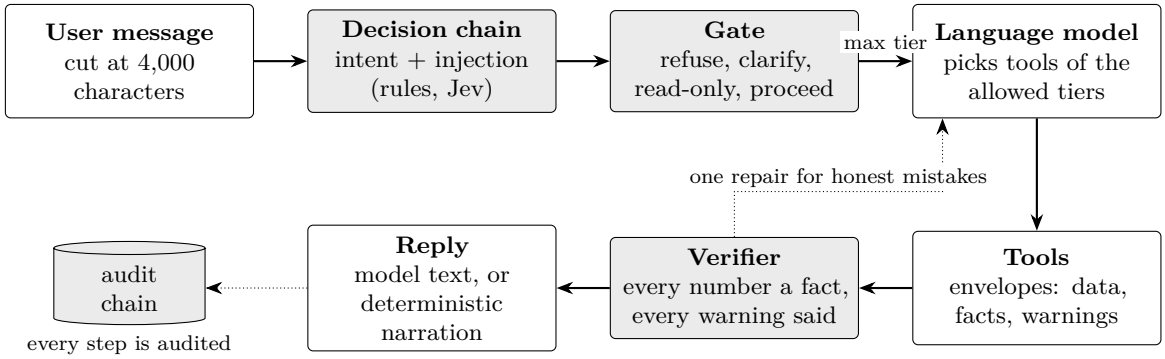


Figure 2: One chat turn. Shaded steps are deterministic code; the language model only selects tools and writes prose.

The tool catalog (`application/catalog.py`) is fixed; its tiers must match the `x-tier` annotation of each contract in `contracts/tools`, which a contract test checks (Table 5). Tool names that would grant a capability the agent must never have (add a beneficiary, transfer funds,

change limits or credentials, override the policy, confirm an action, write the profile or the tolerance) are recognized and blocked as tier T3 with a reason (`PROHIBITED_CAPABILITIES` in `domain/permissions/policy.py`).

Tier	Tools
T0 read	`get_financial_state`, `get_commitments`, `get_user_profile`, `get_pending_actions`, `get_audit_events`
T1 analyze, suggest	`forecast_cashflow`, `simulate_purchase`, `compute_protected_surplus`, `assess_financial_capacity`, `assess_risk_tolerance`, `derive_permitted_profile`, `compute_savings_plan`, `propose_savings_rule` (stateful), `propose_liquidity_floor` (stateful)
T2 execute	`execute_sandbox_action` (only with a user-confirmed authorization)
T3 prohibited	always blocked and audited (`ACTION_BLOCKED`)

Table 5: Agent tools by permission tier. Stateful T1 tools are hidden in read-only turns.

2.5 The confirmation contract

Money can move (in the sandbox) only through three steps, each audited (Figure 3, `application/use_cases/actions.py`). *Propose* creates a pending proposal valid for 30 minutes. A savings rule's amount comes from a stored plan snapshot, never from model text; a liquidity floor's amount is accepted only if the user wrote that number in the same turn (ADR-037). *Confirm* is reachable only from the UI endpoint `POST /v1/actions/proposals/\{id\}/confirm` with the user as actor; the terms shown must equal the proposal's canonical terms. It creates an authorization whose `action_hash` is HMAC-SHA256 of the canonical terms under `NL_HMAC_SECRET`, valid for ten minutes. *Execute* re-reads the terms from that authorization, passes the gate in Algorithm 1, writes `EXECUTION_STARTED` (if the audit cannot be written, nothing executes), consumes the authorization atomically, and only then calls the sandbox bank.

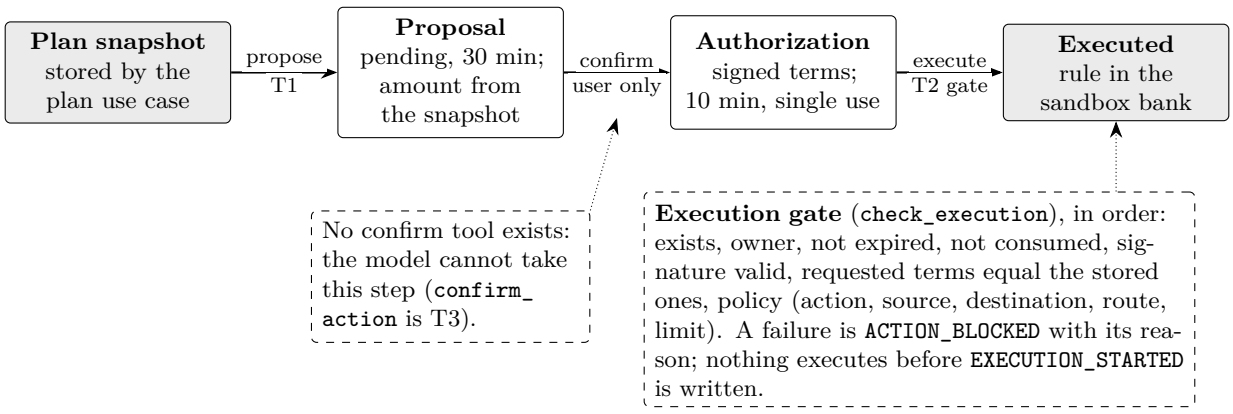


Figure 3: Propose, confirm, execute. Only the user crosses the middle step.

Algorithm 1: Execution gate (`check_execution`, `domain/permissions/policy.py`).

Input: authorization a (or none), caller u, requested terms q (optional), time t, policy P, signature check s

Output: allowed, or blocked with a reason

if *a is none* **then return** blocked `AUTHORIZATION_NOT_FOUND`;
if *$a.user \neq u$* **then return** blocked `WRONG_OWNER`;
if *$t \geq a.expires$* **then return** blocked `AUTHORIZATION_EXPIRED`;
if *a was consumed* **then return** blocked `AUTHORIZATION_REPLAYED`;
if *not s, i.e. HMAC of $a.terms \neq a.action_hash$* **then return** blocked `AUTHORIZATION_TAMPERED`;
if *q given and canonical(q) $\neq$ canonical($a.terms$)* **then return** blocked `AUTHORIZATION_MISMATCH`;
if *action, source or destination not allowed, or (action, source, destination) not a route of P* **then return** blocked (T3);
if *amount > the action's limit (1,000,000 per transfer; 100,000,000 for a floor)* **then return** blocked `AMOUNT_OVER_LIMIT`;
return allowed (T2);

3 Data model

Domain objects are frozen dataclasses; use cases return an `Envelope` (financial reads) or an `ActionOutcome` (actions). Money fields are integers in minor units of the user's currency (CLP has no minor unit, so one peso is one unit).

3.1 The financial model

A `FinancialModel` (`domain/finance/model.py`) is everything the engine needs about one user, either loaded from a persona fixture, estimated from a synthetic ledger (stage 1, `domain/finance/estimation.py`) or estimated from the user's own verified statements (ADR-029, `application/services/statement_model.py`).

Field	Meaning
`as_of`, `first_cycle`	the day before a payday, and the first pay cycle to project (months are pay cycles; payday is the last weekday)
`balances`	checking (the opening trough), reserve, and one bucket per goal
`income`	`typical` and `conservative` monthly income, the coefficient of variation (an exact `Fraction`), and the method (contracted or percentile)
`commitments`	rent, subscriptions, loans and cuotas, each with first and last due cycle and a confidence; a label from merchant text is untrusted data
`spend`	variable spend per category with P50 and P80, an `essential` flag, and seasonal factors (one December factor)
`goals`	target, saved amount and deadline
`source_version`, `largest_one_off_expense`	a content hash of the source data (part of every snapshot id) and the stress shock candidate

Table 6: `FinancialModel`.

3.2 Statements, transactions and actions

A `BankStatement` is what a reader returns: bank layout, masked account reference (`****3456`), currency, period, lines (date, amount, description, kind, section, installment) and everything the bank printed for checking (summary, daily balances, section totals and, for a credit card, its obligations). After redaction and reconciliation it is stored as a `StoredStatement` with its issues; it is *verified* when it reconciles. A verified, final statement is posted line by line as `PostedTransaction`s (`domain/ledger/categorization.py`), each with its category, the classifier that chose it and that classifier's confidence.

`ActionTerms` (`domain/actions/terms.py`) is the exact content of an action: action type (`create_savings_rule` or `set_liquidity_floor`), amount, currency, frequency, source account, destination and effective date, serialized canonically for hashing and signing. A `Proposal` wraps terms with the snapshot it came from, an expiry and a status; an `Authorization` adds the signed `action_hash`, an expiry and, once used, `consumed_at`. Executed rules are `SavingsRule`s in the sandbox bank's store.

3.3 Users, sessions and the audit trail

`UserAccount`
(`domain/user/account.py`): a generated `usr_` id, the normalized email, an encoded scrypt hash and the creation time. The email and the hash never reach the agent, an envelope, a log or the audit.

`Session`
: a generated `sess_` reference (the only session identifier the audit sees), the owner, the SHA-256 digest of the token, creation, expiry (exactly twelve hours later) and revocation time.

`UserProfile`
(`domain/user/profile.py`): display name (1–40 letters, spaces, periods, apostrophes or hyphens, so it cannot carry instructions in markup or code), language, declared tolerance and versioned questionnaire answers. The agent can read it (T0) but never write it: profile writes are user-only HTTP routes.

`AuditEvent`
(`domain/audit/events.py`): an `AuditDraft` (event id, time, session, actor, kind, intent and confidence, decision-model answers, tool and a hash of its parameters, snapshot, policy version, tier, authorization, result, block reason, model version, a short detail, owner) sealed onto a chain with `sequence`, `prev_hash` and `event_hash` = SHA-256 of the canonical JSON of everything else.

3.4 Envelopes and snapshot ids

Every financial read returns an `Envelope` (`domain/shared/envelope.py`, `contracts/common/envelope.schema.json`): a status (`OK`, `INSUFFICIENT_DATA`, `INVALID_INPUT`), the parameter version, the data, a `snapshot_id`, and two lists. `facts` are traceable numbers (key, value, unit, derivation); `warnings` are `Caveat`s with `must_mention` tokens. A snapshot id is a content hash of the model's `source_version`, its `as_of` and the parameter version (ADR-013); with a user floor rule (ADR-037) the floor is part of it too. Changing any number of the input changes the id, so a proposal can always be traced to the exact data and parameters it was sized from. `FinanceParams.fingerprint()` hashes every parameter exactly, and `tests/unit/test_params_lock.py` pins the fingerprint per version: a parameter cannot change without a new `params_version`.

3.5 Persistence schema

The SQLite adapter (`adapters/outbound/persistence/sqlite.py`) keeps schema version 1 in `PRAGMA user_version`. Every column that holds personal or financial content is sealed (Section 6.5); the clear columns are the ones the stores search or order by (Table 7). A file at version 0 (before option B) is upgraded by `upgrade_schema` inside the migration (Algorithm 7).

Writes that change account, session, profile or transaction state commit together with their audit events or not at all: the `*Writes` adapters run one `BEGIN IMMEDIATE` transaction and append the audit drafts inside it, so a failed audit leaves no unrecorded change. `PRAGMA secure_delete=ON` makes SQLite overwrite deleted rows.

Table	Clear columns	Sealed column
statements	statement_id, user_id	payload
transactions	txn_id, user_id, posted_on	payload
user_profiles	user_id	payload
user_accounts	user_id, email_index (HMAC), sample_data	payload
proposals	proposal_id, user_id, session_id	payload
authorizations	authorization_id, user_id, session_id, consumed_at	payload
savings_rules	rule_id, user_id	payload
audit_events	sequence, event_id, user_id, session_id	record
sessions	all: reference, owner, token digest, times	none
meta	name	value (the key check)

Table 7: SQLite schema version 1. The associated data of every sealed value is <table>|<user_id>|<row id>.

4 Financial core algorithms

The engine lives in domain/finance and is pure: the same model and parameters always give the same result. The numbers it uses are the parameter register FC_0_1 in domain/finance/params.py, version fc-0.3 (Table 8).

Docs vs code. docs/FINANCIAL_CORE.md says the register is fc-0.2 "since ADR-034". The code is at fc-0.3: ADR-034 also made a single charge (one cuota) interest-free (interest_free_installments = 1). The golden answers E01–E14 are unchanged across the three versions.

Symbol	Parameter	Value
F	liquidity_floor (or the user's floor rule, ADR-037)	150,000
	stress_shock_min	300,000
	dashboard_horizon_months	6
C	check_horizon_months (plans and purchases)	12
W	plan_window_months	6
	reserve_min_months, reserve_target_months, reserve_rebuild_months	1, 3, 24
	near_term_goal_months	12
$\sigma(p)$	profile_share: conservative, moderate, aggressive	$\frac{2}{5}, \frac{3}{5}, \frac{4}{5}$
	rule_rounding	10,000
	max_installments, interest_free_installments	48, 1
	default_monthly_rate (only when the offer gives none; flagged)	0.025
	min_history_months	6

Table 8: Main parameters of fc-0.3. Amounts in CLP.

4.1 Estimating the model (stage 1)

estimate_model turns a ledger of transactions into a FinancialModel, over pay cycles rather than calendar months, and refuses fewer than six cycles of history. In short (docs/FINANCIAL_CORE.md §4.1–4.3):

- *commitments*: cuotas from structured metadata or from CUOTA nn/NN text, and recurring payments with a 28–35-day cadence and amounts within 5%;
- *income*: the contracted net income, lowered to the recent median when the two differ by more than 5%; for freelance income, the P50 (typical) and P20 (conservative) of cycle totals;
- *variable spend*: P50 and P80 per category, by linear interpolation on exact fractions, net of refunds and excluding cuotas, recurring payments and own transfers; sparse categories

merge into `other`; one December factor, clamped to $[1, 2]$.

The estimators trust the ledger's `transfer_own` label (ADR-012), which is a known risk for real data.

4.2 Forecast at pay-cycle troughs

Liquidity is checked at each cycle's *trough*: the balance just before salary arrives. With B_0 the opening trough, for each projected cycle m:

$$B_m = B_{m-1} + I_m - K_m - V_m - R_m - P_m - S_m, \qquad H_m = B_m - F, \tag{1}$$

where I is income, K commitments due, V variable spend, R a savings rule (the plan), P purchase installments and S a shock. A cycle *breaches* when $H_m < 0$. The *typical* scenario uses typical income and P50 spend; *conservative*, used for every decision, uses conservative income and P80 spend; *stress* is conservative plus one shock $\max(300{,}000, \text{largest one-off expense})$ in the first cycle. The lowest headroom, ties broken by the earliest month, is the binding month (`project` in `forecast.py`, Algorithm 2).

Algorithm 2: Projection (`project`, `domain/finance/forecast.py`).

Input: model M, parameters, scenario $\in$ {typical, conservative, stress}, horizon $h \geq 1$, optional plan, purchases, shock
Output: cycles with trough B_m and headroom H_m

if *scenario is stress and no shock given* **then** shock $\leftarrow \max(\mathit{stress_shock_min}, M.\mathit{largest_one_off})$ in $M.\mathit{first_cycle}$;
$q \leftarrow$ P50 if typical else P80; $I \leftarrow$ typical income if typical else conservative income;
$B \leftarrow M.\mathit{balances.checking}$;
for $m \leftarrow M.\mathit{first_cycle}$ **to** $M.\mathit{first_cycle} + h - 1$ **do**
  $A_m \leftarrow I - M.\mathit{commitments_due}(m) - M.\mathit{spend}(m, q)$; # *available*
  $B \leftarrow B + A_m - \mathit{plan}(m) - \sum \mathit{purchases}(m) - \mathit{shock}(m)$;
  record $(m, B, H_m = B - F)$;

4.3 Protected surplus

The protected surplus over a horizon h is the lowest conservative headroom, with the plan and any purchases included (`protected_surplus` in `surplus.py`):

$$PS(h) = \min_{m \leq h} \Big[B_0 - F + \sum_{k \leq m} (A_k - R_k - P_k) \Big]. \tag{2}$$

It is how much could leave checking today without any conservative trough falling below the floor. A negative value is a breach, not a debt; the engine never calls a positive value "safe".

4.4 Financial capacity

Capacity (`assess_capacity` in `capacity.py`) bands five factors and takes the weakest. Essentials are the P80 of essential categories (groceries, transport, utilities, health) without seasonality, plus average rent and average debt service over the six-month dashboard horizon. With zero essentials every ratio is undefined, so the engine raises `InsufficientData` (hard rule H4) instead of dividing by zero.

The weakest band maps to a profile (low → conservative, medium → moderate, high → aggressive), and every factor tied at that band is reported as binding. The *permitted profile* is the lower of the declared tolerance (from the questionnaire) and capacity (`derive_permitted_profile`); the plan use case recomputes it server-side, so a request for a higher profile is capped and

Factor	Measure	Direction	Bands (a, b)
reserve coverage	reserve / monthly essentials	higher is better	$(1, 3)$
income stability	coefficient of variation of income (sample s.d.)	lower is better	$(0.10, 0.25)$
debt service	peak monthly debt payments (cuotas, loans; not rent) / conservative income	lower is better	$(0.20, 0.35)$
conservative headroom	lowest conservative headroom / floor	higher is better	$(0.25, 1)$
stress coverage	$\max(0, \text{reserve} - \text{stress shortfall}) /$ essentials	higher is better	$(0.5, 1)$

Table 9: Capacity factors. "Higher is better": low below a, high from b. "Lower is better": high below a, low above b.

flagged `PROFILE_CAPPED`. Language never raises a declared tolerance; without one the tolerance is conservative, or at most moderate when the decision model reads a confident signal.

Rule	Behaviour (code)
H1	Reserve below its minimum (1× essentials): the whole rule goes to the reserve (`plan._allocate`); also reported by the permitted profile
H2	No start month gives a positive rule that keeps the conservative path above the floor: no rule (`size_plan`)
H3	Near-term goals (deadline within 12 months) are funded before the reserve top-up, earliest deadline first (`_near_term_goals`); not emitted as a separate code
H4	Unknown state (no income, no spend history, fewer than 6 cycles, zero essentials): `INSUFFICIENT_DATA`, no advice
H5	No sized rule may push a conservative trough below the floor over the check horizon (`size_plan`)

Table 10: Hard rules of the financial core.

4.5 Sizing a savings plan

Algorithm 3 is `size_plan` in `plan.py`. It searches the earliest start month whose rule is positive and keeps the conservative path above the floor. The rule is a share of the lowest available amount in a six-month window, capped by what goals and the reserve top-up actually need, and rounded down to 10,000.

The search is not monotone: a higher profile, or an extra purchase, can produce a smaller rule. `tests/property/test_plan_counterexamples.py` keeps two minimal counterexamples as strict expected failures, and ADR-014 proposes a fix that is not accepted or built.

4.6 Simulating a purchase

A purchase of principal P in n equal cuotas at monthly rate r costs, per cuota (`installments.py`),

$$c = \text{round}\Big(P \cdot \frac{r}{1-(1+r)^{-n}}\Big) \quad (r > 0), \qquad c = \text{round}(P/n) \quad (r = 0), \tag{3}$$

with total cost $n \cdot c$; one cuota carries no interest whatever the rate. `simulate_purchase` overlays the cuotas on the typical and conservative projections over $\max(C, \text{months to the last cuota})$ and returns a verdict: `BASELINE_BREACH` when the conservative path already breaches without the purchase, `FITS_ALL` when the conservative path holds with it, `FITS_TYPICAL_ONLY` when only the typical path holds, else `BREACHES_ALL`. In the last two cases it offers up to three deterministic alternatives (defer the start, another cuota count, a smaller amount), each re-simulated. The

Algorithm 3: Plan sizing (`size_plan`).

Input: model M, parameters, permitted profile p, purchases
Output: a rule amount and start month, its allocation, and the hard rules triggered

$E \leftarrow$ essentials; $target \leftarrow 3E$; $min \leftarrow E$; $s_p \leftarrow \sigma(p)$;
$G \leftarrow$ goals with a deadline within 12 months, by deadline;
$A \leftarrow$ conservative available minus purchases, per cycle, over $C + W$ cycles;
for $i \leftarrow 0$ **to** $C - 1$ **do**
  $s \leftarrow M.first_cycle + i$; $\ell \leftarrow \min_{k<W} A_{s+k}$;
  if $\ell \leq 0$ **then continue**;
  $required \leftarrow \mathrm{round}\big(\max(0, target - reserve)/24\big) + \sum_{g \in G} \lceil (g.target - g.saved)/\max(1, n_{s,g}) \rceil$;
   # $n_{s,g}$: cycles from s to g's deadline
  $R \leftarrow \mathrm{floor}_{10{,}000}\big(\min(s_p \cdot \ell, required)\big)$;
  if $R \leq 0$ **then continue**;
  if *the conservative projection with rule R from s breaches before $s + C$* **then continue** ; *# H5*
  ;
  if $M.reserve < min$ **then**
    return R, s, all to the reserve, H1
  else
    give each goal in G min(remaining, its need); the rest to the reserve;
    return R, s, allocation
return no rule, H2;

largest affordable cuota accounts for cuotas accumulating in the trough: with k_m the number of cuotas paid by cycle m,

$$c_{\max} = \min_{m \geq \text{first cuota}} \lfloor H_m / k_m \rfloor . \tag{4}$$

4.7 Importing a statement

Figure 4 is the import path (`application/use_cases/statements.py`). The reader routes a file by its bytes, not its declared type: an Excel export is parsed with the standard library; a PDF uses its text layer, and a scan or photo needs OCR. Redaction runs right after reading, before anything is stored, classified, audited or returned (ADR-028): deterministic rules in `domain/privacy/redaction.py` replace RUTs (checked with modulus 11), runs of seven or more digits, emails, Chilean phones and the counterparty of a transfer with placeholders such as `[PERSON]` and `[ID]`, and keep only the last four digits of an account. An optional detector can only add masking. If any statement in a file cannot be redacted, nothing from the file is stored.

Reconciliation is the authority over the reader (ADR-020): `reconcile` recomputes totals, section totals, the closing balance and every printed daily balance from the lines, so a misread amount, a dropped line or a wrong day breaks an equality. A statement that does not reconcile is stored as unverified, with a warning naming each issue, and is not posted. A verified, final statement is posted (`post_statement`): on a checking account deposits are income by rule and charges are purchases; a classifier's category is accepted only if it is in the vocabulary with a confidence of at least 0.5, otherwise the charge is `uncategorized`. A category the user sets wins over any model (ADR-023).

4.8 The response verifier

`verify_explanation` (`domain/agent/verification.py`) checks a reply against the facts and warnings of the tools called in the same turn (Algorithm 4). A failed reply is replaced by `narration`, a deterministic text built from the same facts, and the failure is audited as `VERIFICATION_FAILED`.

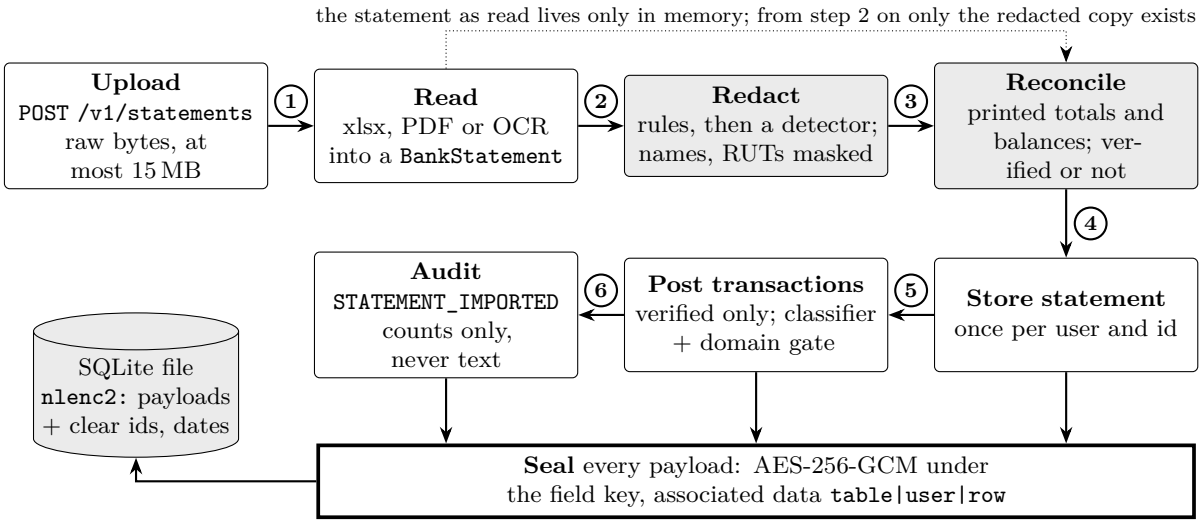


Figure 4: Statement upload. Every payload is sealed before it reaches SQLite; the audit records counts, never text.

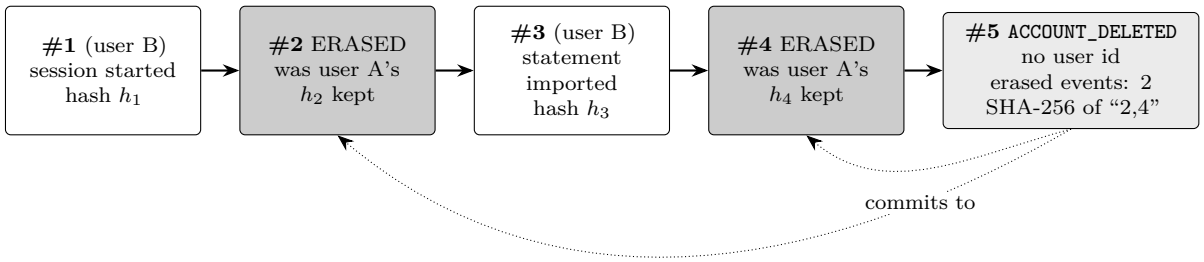
Algorithm 4: Reply verification (`verify_explanation`).

Input: reply text x, facts Φ, warnings Ω, verdicts and execution results of the turn
Output: pass, or the list of issues

foreach *number v in x (currency prefixes, thousands separators, k/mil/M suffixes, percentages)* **do**
  if *v matches no fact exactly, nor to the nearest 1,000 after "about", nor a percentage fact* **then**
    issue `UNSUPPORTED_NUMBER`
foreach *warning $w \in \Omega$* **do**
  if *no token of w.must_mention occurs in x* **then** issue `MISSING_WARNING`;
if *x says "safe money", "safe to spend" or "guaranteed" (English or Spanish)* **then** issue `BANNED_WORDING`;
if *a verdict is not `FITS_ALL` and x affirms the purchase* **then** issue `CONTRADICTION`;
if *x claims a rule was created or executed and no execution succeeded* **then** issue `FALSE_EXECUTION_CLAIM`;
return pass if there is no issue;

4.9 The audit chain and erasure

The audit log is one hash chain for all users (ADR-008). An event's hash covers every field of its record and the previous event's hash, so editing, inserting or deleting an event is detected by `verify_chain`. Account deletion (Section 6.6) must remove a user's events without breaking the chain for everyone else. Each of the user's events becomes a *tombstone*: it keeps its sequence, `prev_hash` and `event_hash`, so the links still verify, and loses everything else (kind `ERASED`, actor system, no user, session `erased`). A tombstone can no longer be re-hashed, so it is accepted only when a later erasure event commits to it (Figure 5, Algorithm 5).



Arrows are the *prev_hash* links. The links and hashes of #2 and #4 are unchanged, so #3 and #5 still verify. A tombstone is accepted only if a later erasure event commits to exactly the set of tombstones that name it.

Figure 5: The chain after user A's account is deleted.

Algorithm 5: Chain verification with tombstones (`verify_chain`, `domain/audit/events.py`).

Input: events $e_1, \ldots, e_n$ by sequence
Output: ok, or the first broken sequence and a reason

$prev \leftarrow 0^{64}$; $T \leftarrow \{\}$ (erasure id $\rightarrow$ tombstone sequences); $X \leftarrow \{\}$ (erasure events);
for $i \leftarrow 1$ **to** n **do**
  if $e_i.sequence \neq i$ **then fail** "sequence gap";
  if $e_i.prev_hash \neq prev$ **then fail** "prev_hash does not link";
  if e_i *is a tombstone naming erasure* j **then**
    if $j \in X$ **then fail** "erased after its erasure event";
    append i to $T[j]$;
  else
    if SHA256(canonical(e_i)) $\neq e_i.event_hash$ **then fail** "event content changed";
  if e_i *is* `ACCOUNT_DELETED` *or* `DEMO_DATA_RESET` **then** $X[e_i.id] \leftarrow e_i$;
  $prev \leftarrow e_i.event_hash$;
foreach $(j, S) \in T$ **do**
  if $j \notin X$ *or* $X[j]$ *does not commit to* $(|S|, \text{SHA256}(S))$ **then fail** "erasure not recorded";
foreach $(j, x) \in X$ **do**
  if x *commits to a count other than* $|T[j]|$ **then fail** "erasure event does not match";
return ok;

The hash is computed over the plaintext record, exactly as `neural-ledger export-audit` writes it, so an export verifies without any key. In the database the whole record, hashes included, is sealed (Section 6.5).

5 Key design decisions

The decision log (`docs/DECISIONS.md`) has 38 records. This section restates the ten that shape the system most, in the log's format: context, decision, consequences. The status is the log's own. Several decisions are still "Proposed" although their code is built and tested: the team marks a record "Accepted" at a sync, and some have not been through one yet.

ADR-001 · Clean architecture with a standard-library core

Status: Proposed.

Context. Five people, ten days, and undecided providers for the language model, the decision model, storage and the client. **Decision.** `domain` and `application` use only the standard library; everything external is a port with an offline adapter; one composition root. Enforced by `tests/architecture`. **Consequences.** A provider swap is one adapter file. Slightly more structure than a script, mitigated by the recipes in `docs/ARCHITECTURE.md`.

ADR-003 · The application orchestrates; the language model is an adapter

Status: Proposed.

Context. An early diagram put the language model in front of everything, in the control path of actions. **Decision.** Use cases own the flow. The decision model classifies the raw text independently of the language model, which only chooses the tools allowed in the turn and writes text. Execution reads terms from the stored authorization. **Consequences.** A fully compromised model cannot move money; test S01 checks it with a hostile fake model.

ADR-004 · A decision-model port of closed questions, in a configurable chain

Status: Proposed.

Decision. The port asks closed questions (choice, score, yes/no) and returns a label, probabilities and a confidence. Adapters: keyword rules (always available) and Jev. `NL_DECISION_CHAIN` sets the order; when every model is unavailable, the turn is read-only. **Consequences.** "Jev first or rules first" is configuration, and no money decision depends on any model.

ADR-006 · Offline mode is a requirement

Status: Proposed.

Decision. The scripted language model and the rules decision model run the whole demo arc without keys or network. **Consequences.** Reproducible CI and recordings, outages covered (S10), and judges can run the system with `docker compose up`.

ADR-008 · HMAC-signed authorizations and a hash-chained audit log

Status: Proposed.

Decision. `action_hash` = HMAC-SHA256(server secret, canonical terms); audit events chained by `prev_hash` and `event_hash`; an execution starts only after its audit event is written. **Consequences.** Tampered terms (S05) and edited audit events are detected; the exported log is evidence anyone can verify. Option B adds tombstones to the chain (ADR-038).

ADR-013 · Content-addressed snapshot ids and a parameter lock

Status: Proposed.

Decision. A ledger's `source_version` hashes its canonical content; every `snapshot_id` hashes `source_version`, `as_of` and `params_version`; `FinanceParams.fingerprint()` is pinned per version in a test. **Consequences.** One peso anywhere changes the snapshot id, and the audit's `forecast_snapshot_id` and `policy_version` always name the data and the numbers used.

ADR-020 · Reconciliation is the authority over the reader

Status: Built.

Decision. The domain recomputes totals and balances from the lines and compares them with what the bank printed. Low OCR confidence only warns. **Consequences.** Amounts are trusted because they add up, never because a reader was confident; descriptions are the only thing reconciliation cannot check.

ADR-028 · Personal data is removed from a statement once, at import

Status: Built.

Context. Statement descriptions carry names, RUTs, account and card numbers, emails and phones. **Decision.** Deterministic rules, plus an optional detector that can only add masking, run before anything is stored, classified, audited or returned. An import that cannot be redacted stores nothing. **Consequences.** No surface (API, audit, database, logs, model) sees the raw text; test S15 checks every surface. Rules are written for Chilean statements, and a name outside a transfer line can slip through.

ADR-037 · The user's own liquidity floor

Status: Built, needs Roles A and B review.

Decision. A floor is an action like a savings rule: proposed by the copilot, confirmed by the user, executed in the sandbox bank. Its amount is accepted only if the user wrote it in the same turn. The latest floor rule replaces the register's 150,000 for that user in every computation, and enters the snapshot id. **Consequences.** `params_version` is unchanged and the golden answers do not move; Role A must confirm that a floor below the register's suits the capacity bands.

ADR-038 · Option B: open sign-up, explicit sandbox, everything sealed, deletable accounts

Status: Built on `security/option-b-hardening`.

Context. Anyone can create an account on the public demo, so a visitor could store a real cartola. Profiles, proposals, rules, the audit chain and the email were plain text, and the offline key file sat on the database's volume. **Decision.** Keep sign-up open, but: a notice that cannot be dismissed, with a server-checked acknowledgement; the key from a Docker secret, refused on the data volume in deployed mode; every personal payload sealed, with HKDF subkeys and an HMAC blind index for the email; account deletion with audit tombstones; demo personas reset, never deleted. It replaces step 6 of ADR-033 (one route instead of two, and erased events instead of kept ones). **Consequences.** One migration per server; losing the key loses the data; the host stays out of scope; a notice is a rule, not a control (Section 6).

Open decisions

- **ADR-014**: plan sizing is not monotone (Section 4.5); a fix is proposed, not accepted.
- **ADR-021**: statements that do not reconcile should go to the user for correction and be usable either way; decided, not built (today they are stored and not posted).
- **ADR-033**: the plan to close the compliance gaps. Rate limits (G07), encryption at rest (G11) and deletion (G03) are built; export (G02), opposition and blocking (G05) and consent (G06) are not.

6 Security design

This section describes the controls as designed and built. The companion *Security self-assessment report* evaluates them, rates the remaining risks and lists what is out of scope.

6.1 Threat model

Asset	Threat	Primary controls
Sandbox money	The model, or text injected through merchant names or chat, moves money	Tiers and the gate; signed, single-use, expiring authorizations only the user can create; terms re-read from storage (Section 2.5)
Truth of advice	The model invents or bends numbers, or hides a caveat	Every number must be a fact, every warning mentioned; otherwise deterministic narration (Section 4.8)
Suitability	A plan more aggressive than the finances allow	Permitted profile = min(tolerance, capacity), recomputed server-side; hard rules H1–H5
Evidence	Audit events edited, removed or forged	Hash chain; erasure only through committed tombstones (Section 4.9)
Identity	A caller claims another user	Cookie sessions resolved server-side; identity header rejected; Origin check (Section 2.3)
Personal data in statements	Stored, returned, logged or sent to a provider	Redaction at import (ADR-028)
Data at rest	A copy of the database, its WAL, a backup or the volume	Every personal payload sealed; key kept off the volume (Section 6.5)
Real data on a sandbox	A visitor uploads a real statement (open sign-up)	Sandbox notice and acknowledgement; sealing; deletion (Section 6.4)

Table 11: Assets, threats and the controls that answer them (`docs/SECURITY.md`).

6.2 Trust boundaries

Figure 6 shows the public demo: a browser reaches Cloudflare's edge over TLS, a tunnel opened from the host carries the request to the web container, and nginx proxies `/api` to the API. Six boundaries matter. **B1** separates untrusted visitors (and their files) from the edge; **B2** is the tunnel into the host; **B3** is the API's own perimeter, where the identity guard, the rate limits and input validation run; **B4** is the data key's separate mount; **B5** is the path to model providers, crossed only in live mode and only by redacted text, and whose answers are untrusted; **B6** is the host itself, outside the scope of the application.

6.3 Agent permissions

The gate (`domain/permissions/policy.py`) knows four tiers: T0 reads, T1 analyzes and proposes, T2 executes an approved sandbox action, T3 is prohibited and always blocked and audited (Table 5). The policy `pp-0.2` allows two actions, each with a fixed route: a savings rule from `checking` to `sandbox_savings` (at most 1,000,000 per action) and a liquidity floor that stays on `checking` (at most 100,000,000). The simulated bank checks the destination again on its side. Text from statements, merchants and the profile's display name reaches the model only wrapped as `{"untrusted_text": ...}`, and profile writes are T3 for the agent, so injected text cannot persist as instructions.

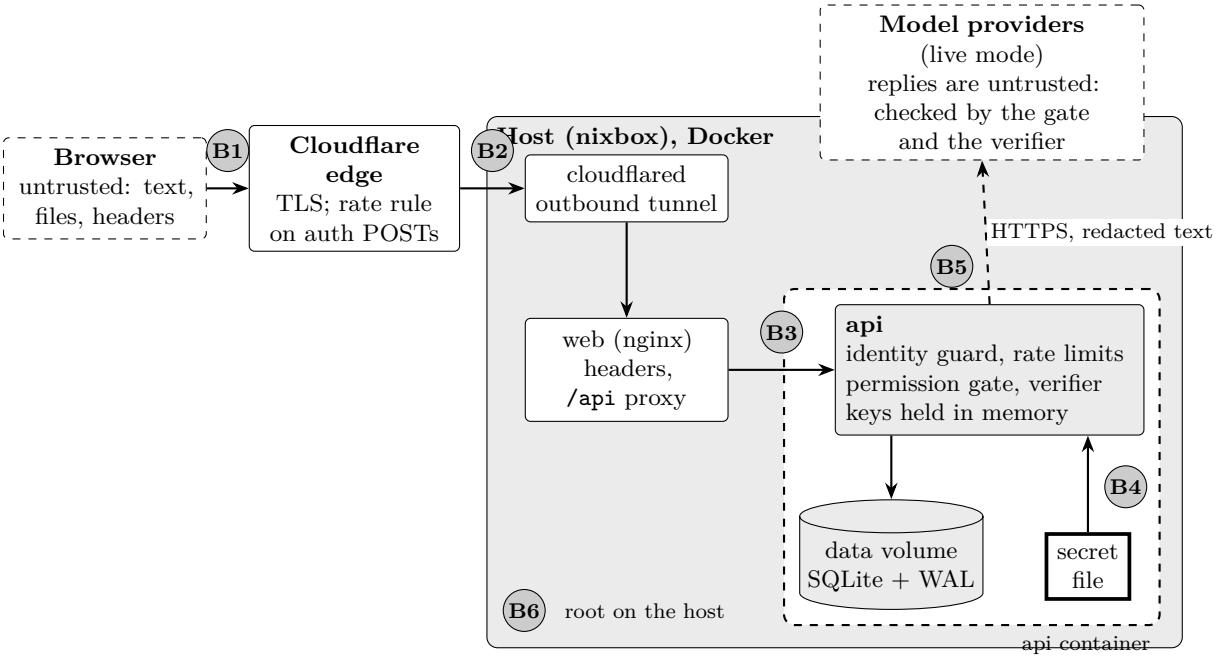


Figure 6: Trust boundaries of the public demo. The deployment override that removes host ports and adds the tunnel is on `ops/deploy-neural-ledger`.

6.4 Accounts, sessions and the sandbox notice

Passwords are hashed with scrypt ($N = 2^{14}$, $r = 8$, $p = 1$, 16-byte salt; `adapters/outbound/system/password.py`); stored parameters are bounded, so a tampered row cannot ask for unbounded memory. A sign-in with an unknown email does the same scrypt work on a dummy hash, and both failures return byte-identical bodies (S20); sign-up, however, answers 409 for a registered email. Sessions are described in Section 2.3.

Sign-up is open (option B). The sign-up page always shows a notice that cannot be dismissed ("This is a hackathon sandbox... do not upload real bank statements and do not enter real personal data") and a required checkbox. The server enforces it: `SignupRequest.sandbox_acknowledged` must be `true`, or the answer is 422 on that field and nothing is stored; `USER_REGISTERED` records which notice was accepted (`sandbox_notice=sandbox-1`). The statement upload screen repeats the warning above the file picker.

6.5 Data at rest

The SQLite adapter seals every payload listed in Table 7 with AES-256-GCM (`FieldCipher` in `adapters/outbound/persistence/field_cipher.py`): a random 12-byte nonce per value, stored as `nlenc2:` followed by base64 of nonce, ciphertext and tag. The associated data is `<table>|<user_id>|<row id>`, so a value copied into another user's row, or into another table, fails to open instead of being read. A value that is not sealed is refused when read: after the migration, plain text in the file can only be an edit made around the application, so it cannot downgrade a field to clear text. The email, which must stay searchable, is replaced by `email_index` = HMAC-SHA256 of the normalized address; the address itself is only inside the sealed account payload. The audit record is sealed whole, its hashes included, so the file holds no hash against which a guessed event could be tested.

6.5.1 Key hierarchy

`NL_DATA_KEY` is 32 random bytes. It is never used directly: HKDF-SHA256 derives one 256-bit subkey per purpose (Figure 7). Values sealed before option B with the raw key (format `nlenc1:`)

still open, and the migration rewrites them. Two other secrets are separate roots: `NL_HMAC_SECRET` signs authorizations, and session tokens and passwords are only ever stored as digests.

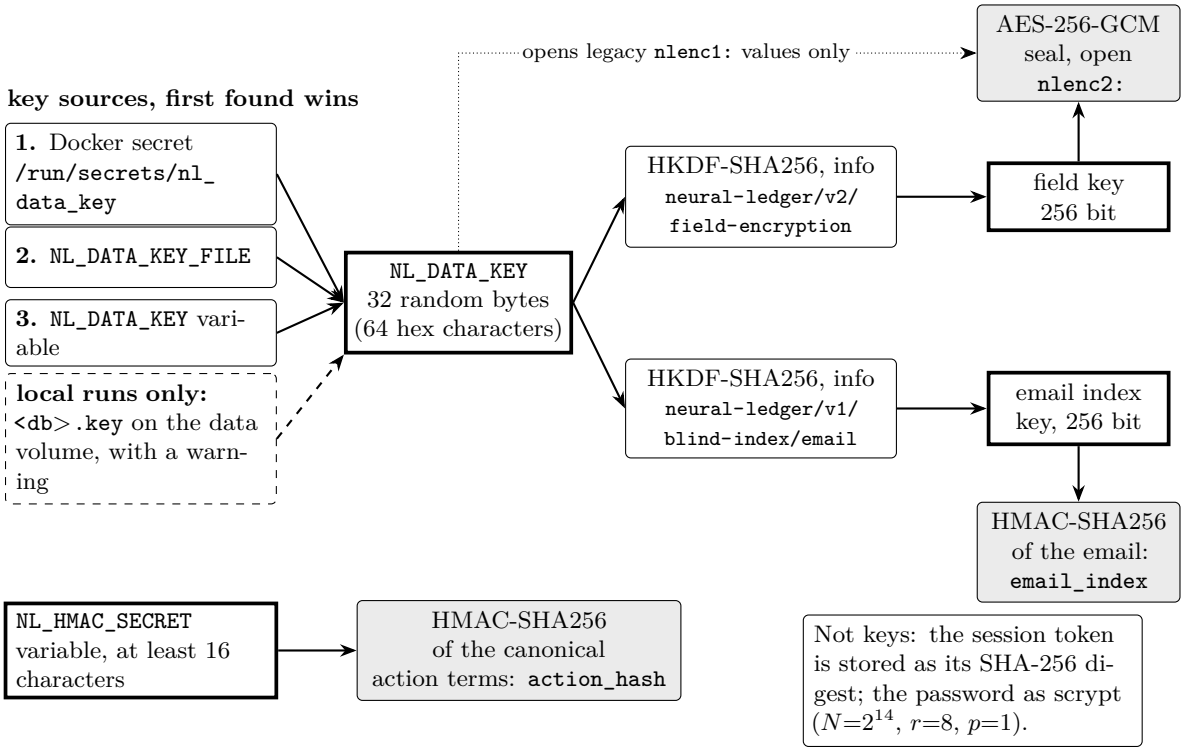


Figure 7: Key hierarchy. Thick-ruled boxes are secret material; shaded boxes are what the keys are used for.

Algorithm 6 is how the API finds the key and what a deployment refuses (`bootstrap/data_key.py`, `_data_cipher` in `bootstrap/container.py`). A refusal happens before the database is opened, so it leaves no file behind. Once a key is chosen, `verify_data_key` opens a sealed known value in the `meta` table (or, in a file that has none, the first sealed value of each column) and refuses a key that does not open it.

6.5.2 Migration and key rotation

`neural-ledger migrate-data` (`bootstrap/migrate.py`, `adapters/outbound/persistence/sqlite_migration.py`) moves a database to the configured key once (Algorithm 7). It runs before the application is built, so the refusals above do not stop it; the operator stops the API first. Pointing `--old-key-file` at the previous key rotates keys the same way.

The transaction makes a failure or a dry run leave the file untouched; running the command again changes nothing; and VACUUM rewrites the file, so free pages keep no copy of the old plain text or old ciphertext. It ends by telling the operator to delete the old key file, which deployed mode refuses to start next to.

6.6 Account deletion

`DELETE /v1/account` takes the password again and a typed confirmation, `DELETE` or the account's email (Figure 8). It deletes every row the user owns, in one transaction, and turns the user's audit events into tombstones that one `ACCOUNT_DELETED` event commits to (Algorithm 8, Section 4.9). Every session ends because its row is gone, and the response clears the cookie. Conversations and snapshots, which live only in memory, are dropped too. The event that records the deletion has no user id and no session, so nothing in the database points at the user any more.

Algorithm 6: Choosing the data key (`resolve` and `check`).

Input: settings S, `/proc/self/mountinfo`
Output: a field cipher, or a refusal

$f \leftarrow S.$*data_key_file* or `/run/secrets/nl_data_key`;
if *f is a file* **then**
  $k \leftarrow$ its content
else if *S.data_key_file was set* **then**
  refuse "not a file"
else if *`NL_DATA_KEY` is set* **then**
  $k \leftarrow$ `NL_DATA_KEY`
else $k \leftarrow$ none;
if *S.deployment is deployed* **then**
  if *k is none* **then** refuse "no data key";
  if *k came from a file on the database's mount (or in its directory)* **then** refuse "key on the data volume";
  if *`<db>.key` still exists* **then** refuse "run migrate-data, then delete it";
  if *`NL_HMAC_SECRET` is the development default* **then** refuse;
if *k is not none* **then return** cipher(k);
if *live mode* **then** refuse "live mode needs NL_DATA_KEY";
warn loudly; **return** cipher(key file `<db>.key`, created once) ; *# local runs only*

Algorithm 7: Migration (`migrate`).

Input: database D, new cipher c', old ciphers O (from `<db>.key` or `--old-key-file`), dry run flag
Output: counts per table: current, re-keyed, sealed from plain text

BEGIN IMMEDIATE;
if *`PRAGMA user_version` < 1* **then**
  rebuild `user_accounts` (email → `email_index`), and `proposals`, `authorizations`, `audit_events` (add `user_id`)
foreach *sealed column* (t, aad) *and row* r **do**
  $v \leftarrow r.value$;
  if *v is not sealed* **then**
    $x \leftarrow v$; *# plain text from before*
  else if *c' opens v* **then**
    if *v is `nlenc2:`* **then** count current; **continue**;
    $x \leftarrow c'$.open(v)
  else if *some $c \in O$ opens v* **then**
    $x \leftarrow c$.open(v)
  else ROLLBACK; refuse "opens with no given key";
  write c'.seal(x, $aad(r)$);
write the key check sealed with c';
if *dry run* **then**
  ROLLBACK
else
  COMMIT; VACUUM; WAL checkpoint (truncate)

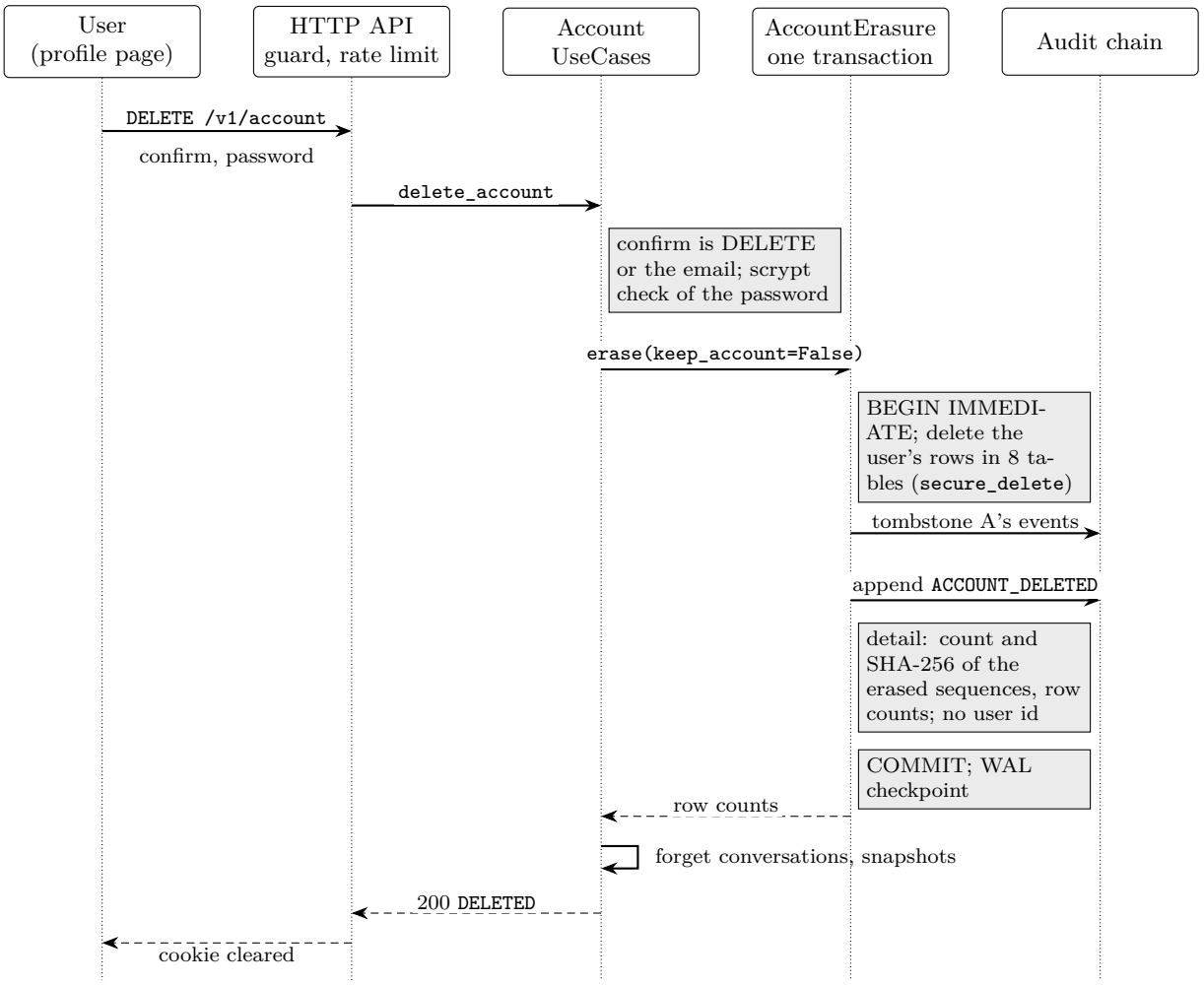


Figure 8: Account deletion. One SQLite transaction covers every row and the audit chain.

Algorithm 8: Erasure (`SqliteAccountErasure.erase` in `sqlite.py`).

Input: user u, keep_account (true for a demo persona's reset), erasure draft d with no user id
Output: rows deleted per table, number of tombstones

BEGIN IMMEDIATE;
foreach *table in statements, transactions, savings_rules, proposals, authorizations* **do**
  delete rows of u; count
if *not keep_account* **then** delete u's sessions, profile and account; count;
$E \leftarrow \emptyset$;
foreach *audit event e of u, by sequence* **do**
  if *e is an erasure event* **then continue**;
  replace e with its tombstone naming d (same sequence, `prev_hash`, `event_hash`); add *e.sequence* to E;
append d with detail `erased_events=`$|E|$`; erased_sha256=SHA256(`E`); counts`;
COMMIT; WAL checkpoint (truncate);

Demo personas are reset, not deleted. Camila and Diego are shared fixtures with no password. In a persona session the profile page offers *Reset demo data*: confirmation `RESET`, no password, and the erasure above with `keep_account` true, recorded as `DEMO_DATA_RESET`. The fixture and every visitor's session stay.

6.7 Evidence

`make test-security` runs the attack tests S01–S23; `docs/SECURITY.md` lists each with its expected behaviour and audit event. Option B adds `tests/adapters/test_data_key.py` (key sources, deployed refusals, wrong key, migration), `tests/adapters/test_encryption_at_rest.py` (raw database bytes after a full HTTP flow; upgrade of a schema 0 file) and `tests/security/test_account_deletion.py` (user B intact, no row of any table refers to user A, forged tombstones fail). `tests/security/test_compliance.py` turns each legal requirement into a test that names its norm (Section 7).

7 Privacy and Law 21.719

Chile's Law 19.628, as amended by Law 21.719 (published 2024-12-13, in force 2026-12-01), governs personal data. Bank statements are personal and financial data, and profiling a person's finances is likely high-risk processing. This section maps what the software does to the law's main duties. It is engineering evidence for counsel to review, not legal advice; article numbers follow the Library of Congress summary that `docs/SECURITY.md` cites.

7.1 What is processed, where and for how long

Data	What is kept	At rest
Statements	Bank, period, each line's date, amount, kind and redacted description, balances, card obligations; names, RUTs and long numbers replaced by placeholders; account as its last four digits	sealed
Transactions	Date, amount, redacted merchant, category, the classifier and its confidence	sealed; date clear
Account	Email (found by an HMAC index), scrypt hash, creation time	sealed
Profile	Display name, language, questionnaire answers, declared tolerance	sealed
Proposals, authorizations, rules	Terms, amounts, signature, status, expiry	sealed
Audit trail	What happened, by whom (ids), when; counts, never statement text; no names, RUTs, emails or card numbers (C10)	sealed
Sessions	Reference, owner, token digest, times	clear (no content)
Conversations, snapshots	The last 40 chat entries per session; computed results	memory only

Table 12: Data inventory (`docs/PRIVACY.md`).

There is no automatic retention period: data stays until the user deletes the account or the operator deletes the database. With the default settings nothing leaves the server. A provider receives data only when its setting is on (live mode): the language model receives the conversation and tool results; Jev receives chat messages and, for categories, redacted charge text, never a deposit; Azure AI Language receives raw line text before redaction. All of them are outside Chile.

7.2 Duties and rights

Each built control (C01–C13 and the G rows that are built) is a plain test, and each missing duty is a strict expected failure marked `GAP`: CI stays green, the gap stays visible, and the day a control

Duty or right	Article	In the software
Proportionality, minimization	3 c)	Redaction at import (C01, C02); card numbers kept as four digits
Access and portability	5, 9	Partly: statements, transactions and audit events are readable by their owner; no full export (gap G02)
Correction	6	Partly: a transaction's category (G04); line corrections are ADR-021, not built
Deletion	7	Built (G03): `DELETE /v1/account`, audit tombstones (Section 6.6)
Opposition, blocking	8, 8 ter	Not built (G05)
Consent	12	Not recorded per purpose (G06); sign-up records the sandbox notice only
Information	14 ter	`docs/PRIVACY.md` (G12)
Security	14 quinquies	Encryption at rest (G11), key off the volume, sessions, rate limits (G07), headers (G09)
Breach reporting	14 sexies	Hash-chained audit (C10); incident runbook (G12)
Impact assessment	15 ter	Not written; required before real users
Children	16 quáter	No age check; users are assumed adults
International transfer	27	Off in offline mode; safeguards for live providers not documented

Table 13: Law 21.719 duties and the software (`tests/security/test_compliance.py`).

lands its test passes unexpectedly and the run fails until the marker is removed. `test_every_compliance_test_is_in_the_security_map` keeps `docs/SECURITY.md` and the tests in step.

7.3 The sandbox posture

The public demo is a sandbox with open sign-up. Its privacy position rests on four things: the notice asks visitors not to store real data and records that they agreed; whatever is stored anyway is redacted at import and sealed at rest; its owner can delete it; and the operator has an incident runbook for real data found on the server (`docs/runbooks/INCIDENT_RESPONSE.md`). None of this prevents a visitor from uploading a real statement, and data uploaded under a shared persona is visible to every other visitor of that persona until someone resets it. Before any use with real users, the open gaps (G02, G05, G06), the impact assessment and the transfer safeguards must be closed.

8 Where the Markdown documentation and the code disagree

Where the files in `docs/` and the README disagree with the code at commit `58ff160`, this document follows the code. The differences found while writing it:

1. **Parameter version.** `docs/FINANCIAL_CORE.md` gives the register as `fc-0.2`; the code is at `fc-0.3` (Section 4).
2. **Names of policy objects.** The constants `FC_0_1`, `AP_0_1` and `PP_0_1` hold versions `fc-0.3`, `ap-0.2` and `pp-0.2`. The audit records the version string, not the constant's name.
3. **Forecast from a user's own statements.** `docs/PRIVACY.md` ("Why") says the forecast and the plans compute from the persona model and that ADR-029 is not built. In the code, an account's model is estimated from its own verified statements (`StatementModelSource`, ADR-029 step 2); personas keep their fixtures.
4. **Plain-text payloads.** ADR-033 step 2 says payloads written before encryption "are still read". Since option B a value in clear is refused; an older file is upgraded and sealed at startup or by `migrate-data`.

5. **Status of the security suite.** The README's status list and document table mention S01–S12 and S01–S15; `docs/SECURITY.md` and the tests cover S01–S23 plus the option B tests.

6. **"No accounts or personal data."** The README ends with "no real money, accounts or personal data". Accounts exist in accounts mode, and with open sign-up the software cannot guarantee that no personal data is stored; `docs/SECURITY.md` now says so.

7. **Runtime modes.** The table in `docs/ARCHITECTURE.md` does not list `NL_DEPLOYMENT` (Table 3).

8. **Deployment files.** `docs/SECURITY.md` refers to the public demo's server and its deployment override; that override, its nginx configuration and its runbook exist only on the unmerged branch `ops/deploy-neural-ledger`, so what this document says about the edge (TLS, the tunnel, nginx headers) comes from that branch, not from the commit described.

9. **Decision statuses.** Several records in `docs/DECISIONS.md` are "Proposed" although their code is built and tested (Section 5).